

LAPORAN PRAKTIKUM PEKAN 4
PEMOGRAMAN BERBASIS OBJEK
“Ekspansi Produk Keuangan (Inheritance)”

disusun Oleh:

Nabil Fikri (2511533011)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M. Dawi Syauqi



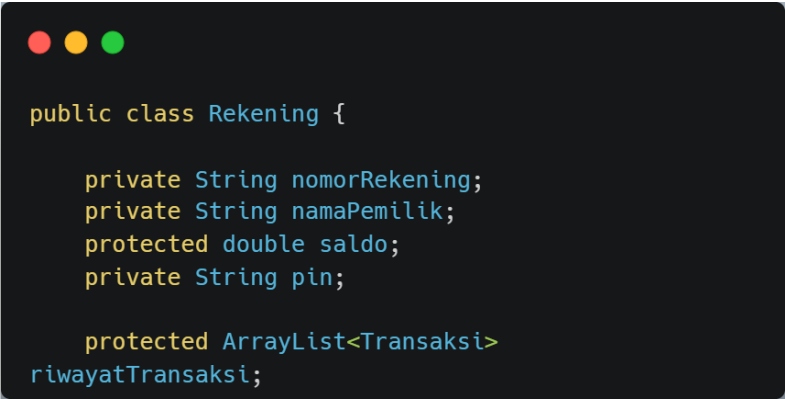
DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

A. ISI PRAKTIKUM

Praktikum Pekan 4 bertujuan untuk memahami dan menerapkan konsep **Inheritance (Pewarisan)** dalam Java melalui studi kasus sistem perbankan multi-produk, meliputi penggunaan keyword `extends` dan `super`, modifier `protected`, serta Upcasting dan Downcasting.

1. Modifikasi Rekening.java



```
public class Rekening {  
  
    private String nomorRekening;  
    private String namaPemilik;  
    protected double saldo;  
    private String pin;  
  
    protected ArrayList<Transaksi>  
    riwayatTransaksi;  
}
```

Class `Rekening` merupakan kelas induk(Superclass) dari class `RekeningTabungan` dan `RekeningGiro`. Pada potongan gambar di atas terlihat bahwa modifier atribut `saldo` dan `ArrayList<Transaksi> riwayatTransaksi` berubah dari `private` ke `protected` agar subclass dapat mengaksesnya langsung tanpa melanggar enkapsulasi ke `public`.

2. Membuat Subclass RekeningTabungan

```
package Pekan4;

public class RekeningTabungan extends Rekening{

    // atribut spesifik yang hanya dimiliki oleh tabungan
    private double sukuBunga;
    public double getRek() {
        return sukuBunga;
    }
    public void setRek(double sukuBunga) {
        this.sukuBunga = sukuBunga;
    }

    // konstruktor subclass
    public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal, double sukuBunga)
    {
        // super() memanggil konstruktor kelas induk (Rekening). WAJIB berada di baris pertama!
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        // Menghitung Bunga
        // mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan?
        double nominalBunga = saldo * (sukuBunga / 100);
        saldo += nominalBunga;

        // mencatat riwayat transaksi
        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));

        System.out.println("Bunga" + sukuBunga + "% berhasil ditambahkan Rp" + nominalBunga);
    }
}
```

Tambahkan class RekeningTabungan.java(Subclass) yang memiliki atribut sukuBunga dan method tambahBungaAkhirBulan().

- Kode extends Rekening pada RekeningTabungan mewarisi semua atribut dan method milik Rekening.
- Kode super(nomor, nama, saldoAwal, pinAwal) berfungsi untuk memanggil konstruktor induk. Ini wajib berada di baris pertama konstruktor anak.
- Atribut saldo dan riwayatTransaksi dapat diakses langsung karena modifier protected.

3. Membuat Subclass RekeningGiro

```
package Pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double
saldoAwal, String pinAwal, double batasOverdraft) {
        // memanggil inisialisasi dasar dari Superclass
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
    }

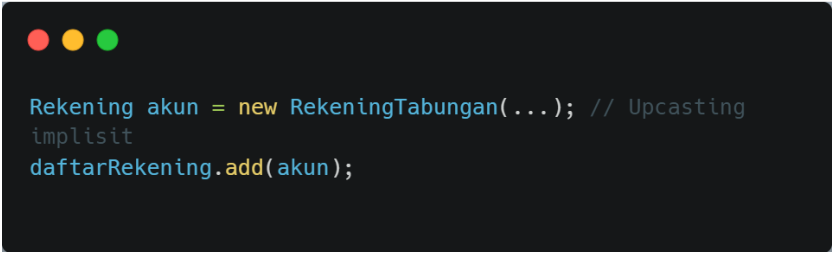
    // getter khusus Giro
    public double getBatasOverdraft() {
        return batasOverdraft;
    }

    // catatan : penarikan hingga limit overdraft akan
    kita selesaikan di modul 5
}
```

Setelah membuat class RekeningTabungan, selanjutnya membuat class RekeningGiro (Subclass) yang juga merupakan anak dari Rekening.java dengan tambahan atribut batasOverdraft (limit pinjaman)

- RekeningGiro mewarisi semua fungsi rekening dasar (setor, tarik, cek saldo) dari superclass Rekening.
- Menambahkan atribut spesifik batasOverdraft yang hanya dimiliki produk Giro.

Konsep Upcasting :



```
Rekening akun = new RekeningTabungan(...); // Upcasting  
implicit  
daftarRekening.add(akun);
```

Saat menyimpan objek subclass ke `ArrayList<Rekening>` `daftarRekening`, terjadi **Upcasting** secara otomatis. Ini memungkinkan satu koleksi menampung berbagai jenis rekening sekaligus.

B. Latihan/Tugas

Tugas 1: Pembuatan Multi-Produk (40%)

Modifikasi case 1: Buka Rekening Baru di `Main.java`.

1. Setelah user memasukkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek `RekeningTabungan`.
3. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek `RekeningGiro`.
4. Simpan objek tersebut ke dalam `daftarRekening` (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

Jawaban :

Output :

Jika user memilih Tabungan Umum:

```
Pilih Menu : 1
Masukkan No Rekening : 111
Masukkan Nama Pemilik : nnn
Masukkan Saldo Awal : 100000
Masukkan PIN : 123456
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
3. RekeningVIP
Pilihan : 1
Masukkan Suku Bunga (%) : 10
Rekening atas nama nnn berhasil dibuat dengan saldo Rp100000.0
```

Jika user memilih Giro Bisnis:

```
Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
3. RekeningVIP
Pilihan : 2
Masukkan Batas Overdraft : 100000
Rekening atas nama bbb berhasil dibuat dengan saldo Rp100000.0
```

Variabel rekeningBaru bertipe Rekening (superclass), namun menampung objek RekeningTabungan atau RekeningGiro (subclass). Java mengenali ini secara otomatis karena ada relasi Is-A (Tabungan *is-a* Rekening).

Tugas 2: Simulasi Waktu Akhir Bulan : 60%)

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().

4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

Jawaban;

```
System.out.println("7. Simulasi Akhir Bulan (Khusus  
Tabungan)");  
System.out.println("0. Keluar");  
System.out.print("Pilih Menu : ");
```

```
case 7 :  
    if (akunAktif == null) {  
        System.out.println("Error: Anda belum  
membuka rekening!");  
    } else if (akunAktif instanceof  
RekeningTabungan) {  
        // Downcasting: Rekening ->  
RekeningTabungan  
        RekeningTabungan tabungan =  
(RekeningTabungan) akunAktif;  
        tabungan.tambahBungaAkhirBulan();  
    } else {  
        System.out.println("Gagal: Fitur bunga  
akhir bulan hanya berlaku untuk Rekening Tabungan.");  
    }  
    break;  
  
case 0 :  
    isRunning = false;  
    System.out.println("Sistem ditutup. Terima  
Kasih!");  
    break;  
  
default:  
    System.out.println("Pilihan tidak  
valid!");  
}
```

Output :

```

Pilihan : 1
Masukkan Suku Bunga (%) : 10
Rekening atas nama nnn berhasil dibuat dengan saldo Rp100000.0
Rekening berhasil dibuka atas nama nnn!

Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu : 7
Bunga10.0% berhasil ditambahkan Rp10000.0

```

- akunAktif bertipe Rekening (superclass), sehingga method tambahBungaAkhirBulan() tidak bisa dipanggil langsung karena method itu hanya ada di RekeningTabungan.
- Operator instanceof **wajib digunakan lebih dulu** untuk mencegah ClassCastException saat runtime.
- Setelah dipastikan tipenya benar, barulah dilakukan cast eksplisit: (RekeningTabungan) akunAktif.

C. Analisis dan Kesimpulan

1. Analisis

Pada praktikum pekan 4 ini, konsep utama yang diterapkan adalah **Inheritance (Pewarisan)** dalam Java. Rekening.java dijadikan superclass dengan mengubah modifier atribut saldo dan riwayatTransaksi menjadi protected, sehingga subclass dapat mengaksesnya secara langsung tanpa melanggar prinsip enkapsulasi. Dari superclass tersebut, dibuat dua subclass yaitu RekeningTabungan dan RekeningGiro menggunakan keyword extends. Masing-masing subclass memanggil super() di baris pertama konstruktornya untuk menginisialisasi bagian yang diwarisi dari Rekening, kemudian menambahkan atribut spesifik miliknya sendiri seperti sukuBunga pada RekeningTabungan dan batasOverdraft pada RekeningGiro.

Pada bagian latihan, konsep **Upcasting** diterapkan saat menyimpan objek subclass ke dalam `ArrayList<Rekening>`. Proses ini terjadi secara implisit dan aman karena adanya relasi Is-A antara subclass dan superclass. Sebaliknya, konsep **Downcasting** diterapkan pada menu Simulasi Akhir Bulan, di mana referensi bertipe `Rekening` perlu dikembalikan ke tipe spesifik `RekeningTabungan` agar method `tambahBungaAkhirBulan()` dapat dipanggil. Sebelum melakukan downcasting, pengecekan tipe menggunakan operator `instanceof` wajib dilakukan untuk mencegah terjadinya `ClassCastException` saat program berjalan.

2. Kesimpulan

Melalui praktikum ini dapat disimpulkan bahwa Inheritance memungkinkan pengembangan sistem yang lebih efisien dengan menerapkan prinsip **Code Reusability**, yaitu kode yang sudah ada pada superclass tidak perlu ditulis ulang di setiap subclass. Dengan mewarisi `Rekening` sebagai superclass, sistem perbankan dapat dengan mudah diperluas menjadi berbagai produk rekening tanpa menduplikasi logika yang sama. Selain itu, kombinasi Upcasting dan Downcasting memberikan fleksibilitas dalam mengelola koleksi objek yang beragam dalam satu tipe seragam, sekaligus tetap memungkinkan akses ke fitur spesifik masing-masing subclass apabila diperlukan.

D. Link Repository

https://github.com/nabilfkrr/PrakPBO_C_2511533011/tree/PrakPBO_Pekan4/Pekan4

